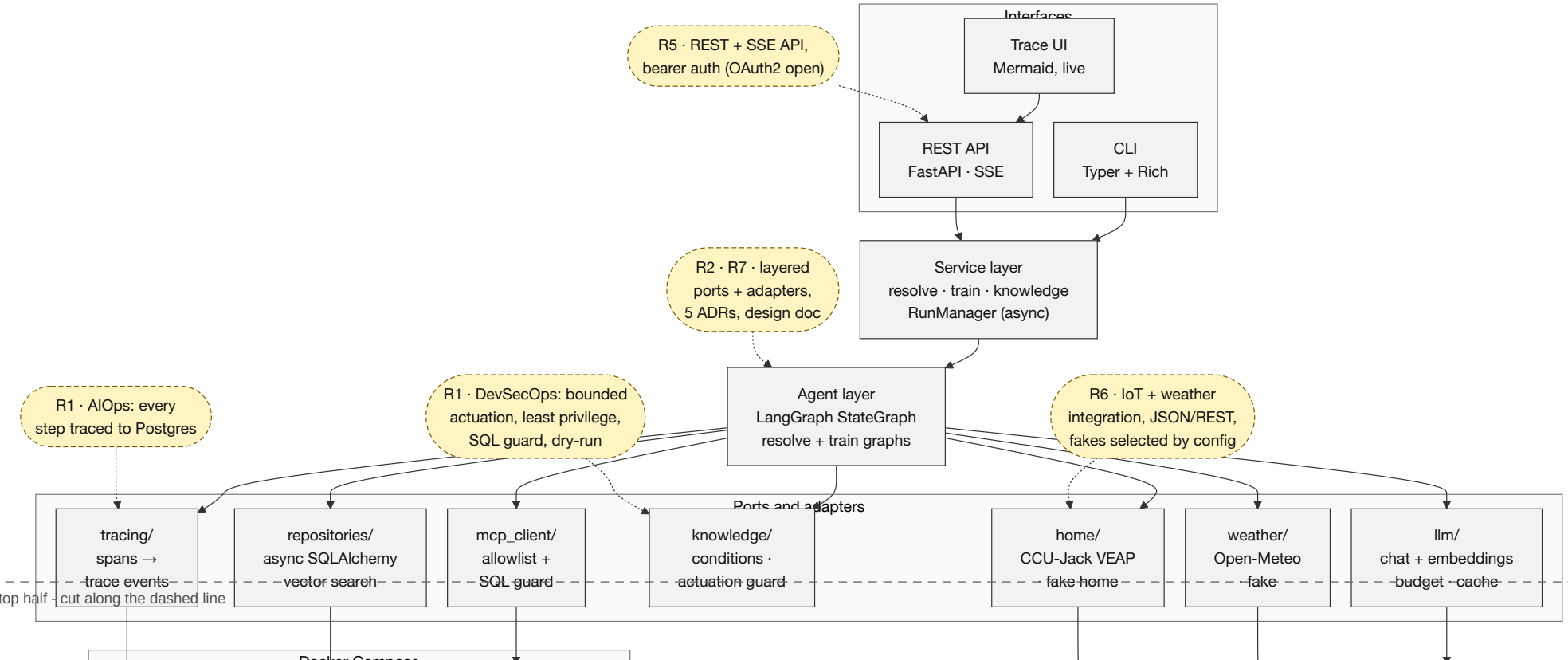
Buddler: Agentic home automation, mapped to the IBM AI Architect profile

A sentence like "*I'm feeling cold*" is turned into an observed, knowledge-backed decision for a Homematic home: the agent reads room, clock and weather, matches curated symptom → action knowledge (Postgres/pgvector) and then acts on the thermostat within hard bounds, or advises. Every step is traced.

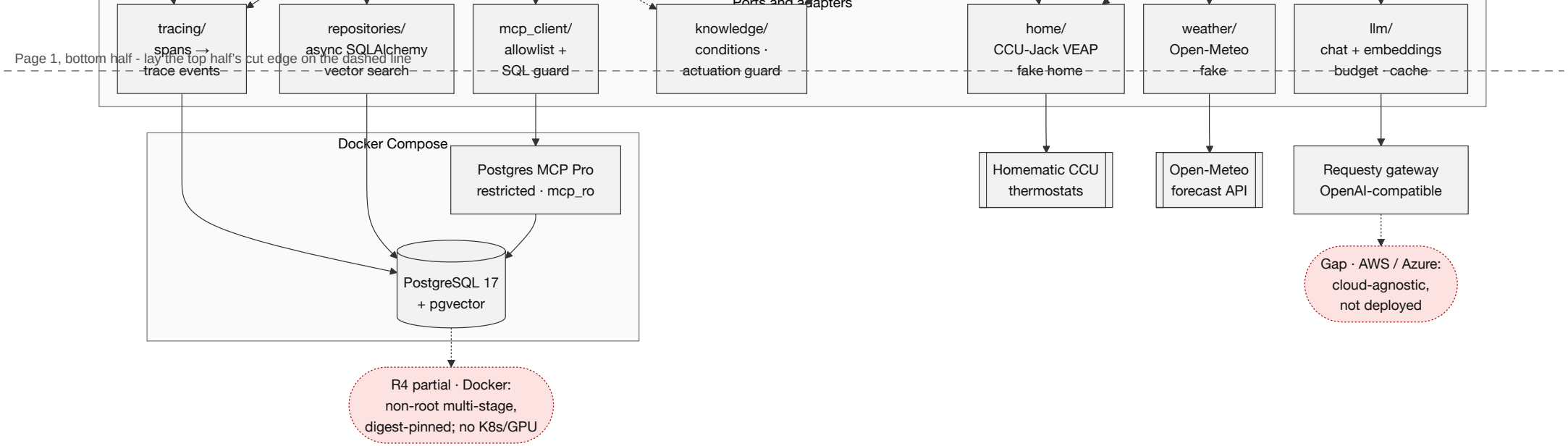
R1 AI/ML foundations, DevSecOps & AIOps
R2 Design of complex IT architectures with AI components
R3 LLMs, generative & agentic AI, robust and scalable
R4 Containers + GenAI/agentic architectures (RAG, patterns)
R5 DevSecOps/MLOps, API design & integration, Python
R6 Data formats, sources, interfaces, end-to-end flow
R7 System engineering: decompose and recompose systems
R8 Performance and scalability of analytics components

Legend: system component · requirement covered (balloon) · requirement not covered / partial

1 · Architecture

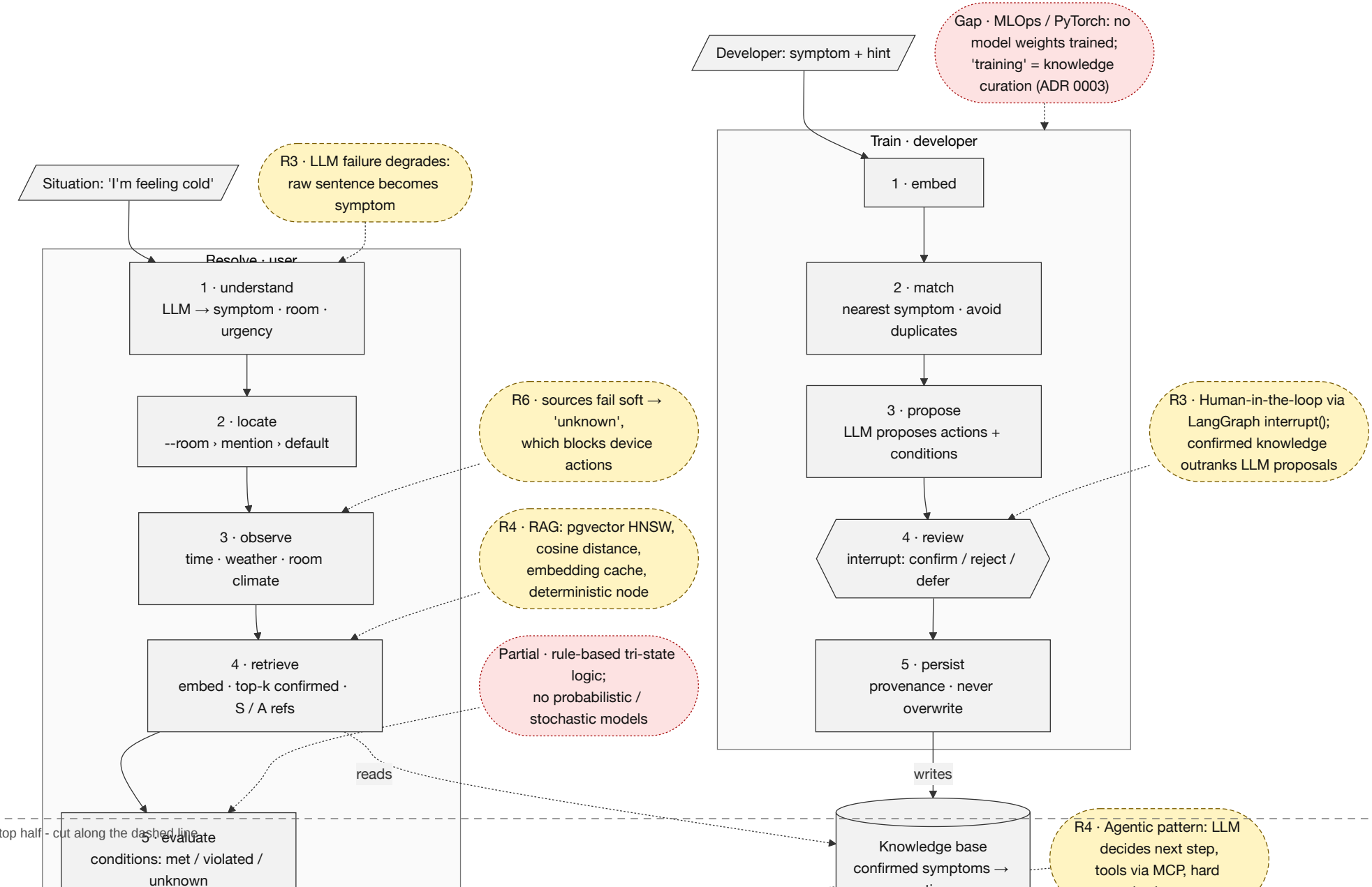


Page 1, bottom half - lay the top half's cut edge on the dashed line

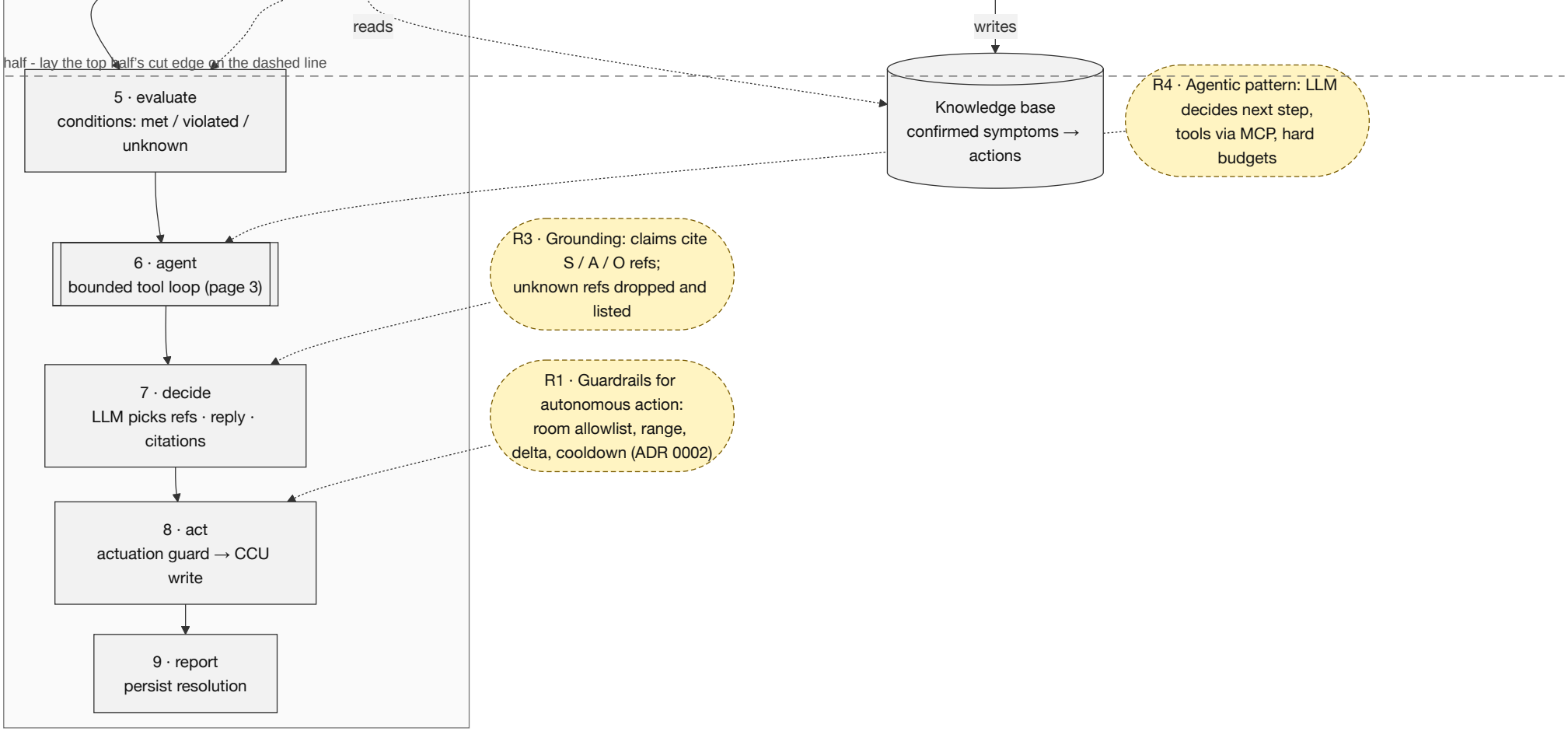


2 · Workflows (LangGraph): Resolve acts or advises, Train curates knowledge

Resolve mixes LLM steps (understand, agent, decide) with deterministic ones (observe, retrieve, evaluate, act). The LLM only chooses among pre-evaluated candidates by ref; device parameters come from confirmed knowledge. Train *curates* knowledge with a human in the loop; Resolve only reads it.

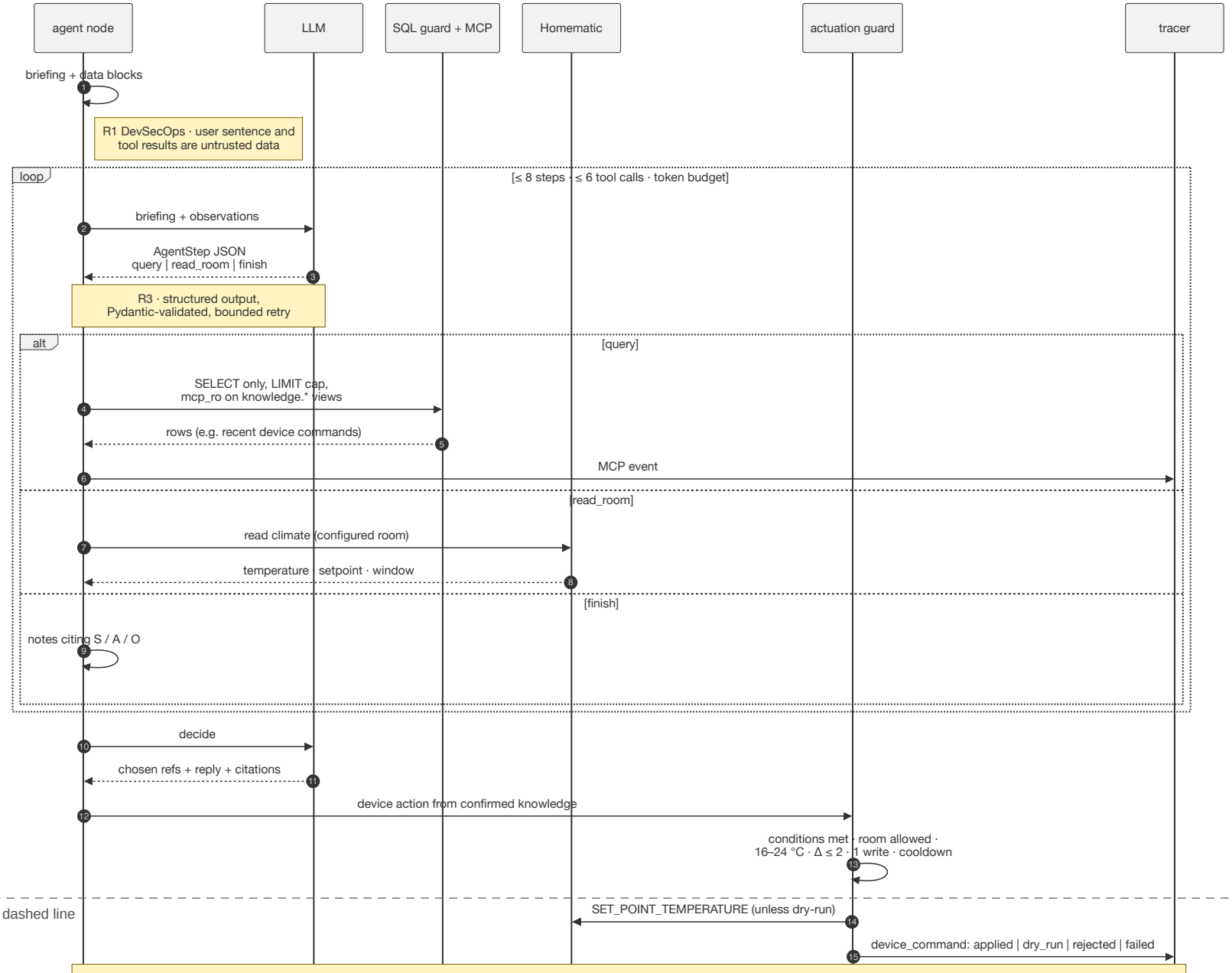


Page 2, bottom half - lay the top half's cut edge on the dashed line

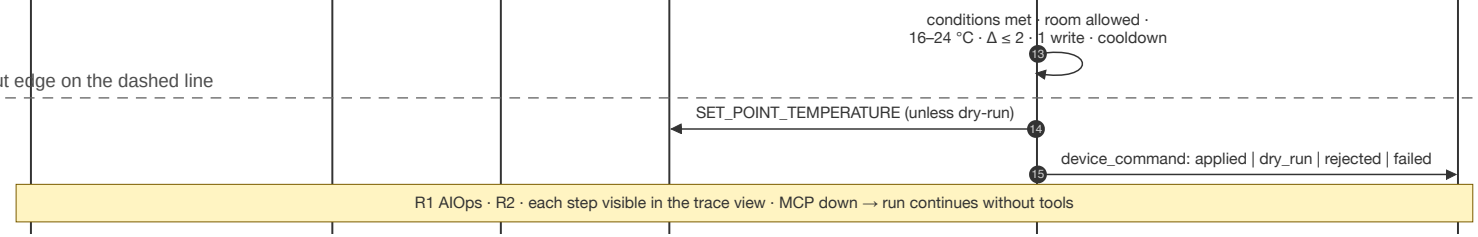


3 · Agentic AI: bounded tool loop and guarded actuation (Resolve, steps 6–8)

Structured-output loop on the `llm/` port: each step is one validated JSON decision, `query` (SQL on curated read-only views via MCP), `read_room` (climate of another room) or `finish`. The agent has no device-write tool: writes happen only in `act`, after the guard. Notes in yellow mark the requirements shown at that point.



Page 3, top half - cut along the dashed line



Page 3, bottom half - lay the top half's cut edge on the dashed line

Coverage summary

Requirement	Evidence in this project	Status
R3 LLMs, generative + agentic AI	LangGraph workflows, bounded tool loop, structured output, grounded citations, autonomous but guarded actuation	**Strong**
R4 RAG / agentic patterns	pgvector retrieval, tool use via MCP, human-in-the-loop interrupt, LLM chooses only among evaluated candidates	**Strong**
R2 · R7 Architecture, decomposition	Ports and adapters (LLM, home, weather, MCP), design doc, 5 ADRs, fakes per port	**Strong**
R5 API design, Python	FastAPI REST + SSE, OpenAI-compatible gateway, MCP; Python 3.12, mypy strict, ruff, ~95 pytest tests	**Strong**
R6 Data sources, interfaces	CCU-Jack VEAP (IoT), Open-Meteo, Postgres; JSONB contracts validated by Pydantic	**Strong**
R1 DevSecOps, AIOps	Actuation guard, least-privilege DB roles, SQL guard, dry-run, digest-pinned images; full tracing	**Good**
R8 Performance, scalability	HNSW index, embedding cache, async I/O; small data volume, no load tests	**Partial**
R4 Containers / OpenShift, GPU	Docker Compose, non-root multi-stage image	**Partial**
R5 CI/CD, MLOps, PyTorch	Local quality gates only; no pipeline, no model training	**Gap**
Cloud AWS / Azure, hybrid	Cloud-agnostic gateway; no cloud deployment	**Gap**
Probabilistic / stochastic models	Deterministic condition rules, vector similarity, weather forecast as input	**Gap**

Source: docs/design.md, docs/adr/*, AGENTS.md, src/buddler/agents/graphs/*. Requirement wording condensed from the job advertisement.